## Spring 2025 Semester Summary

This semester, we focused on developing tools for energy-efficient AI profiling. We integrated Scalene for line-by-line profiling and implemented joule measurement based on TDP to estimate CPU and GPU power consumption. However, extensive testing was limited due to a lack of hardware access. We successfully created a presentation summarizing our progress and gained access to servers for future testing. While we did not develop a UI or ML model this semester, we laid the groundwork for further testing and development in future semesters.

## Project Goals

**Short-Term:**

- Continue testing our current Scalene implementation on our newly acquired Chameleon servers, as they offer significantly more compute power. This will be used to enhance our Scalene Python profiler.
- Plan and begin development on our ML-driven Refactoring tool
- Expand the languages that can be profiled to Java. This may require a completely different profiler than Scalene.
- Test and validate the accuracy and precision of our systems

**Long-Term:**

- Have an executable program that first profiles AI training programs/files, identifies and records methods and lines of high energy consumption, and then provides an ML-driven refactoring recommendation to reduce the energy consumption.

**Connection:**

- The advancement of the Python profiler and the development of our ML-driven refactoring tool will directly feed into our long-term goal of having a fully fledged tool. These two aspects are the basis of this project.
- Expanding the languages that can be profiled won't directly support our long-term vision, but it will expand the coverage of AI programs that can be profiled by our eventual tool.
- Validating the accuracy and precision of our profiling and refactoring systems is a necessary step in creating an applicable tool for actual AI programmers.

## Tech Stack

**Languages:** Python, Java
**Libraries & Repos:** Scalene, LibCST

## Problem Space

Artificial Intelligence and Machine Learning have been two topics at the forefront of development in recent history. The advancement and use of these two have reached the point

where millions continue to benefit from them. However, the energy consumption from the use and training of AI and ML models is continuing to climb to staggering heights. This project aims to be a direct solution to the immense energy needed to train AI. While there exists partial solutions or tools to help identify areas of high energy consumption, our project differs in its attempt to take those areas and generate recommendations for AI developers to reduce their energy consumption.

## Demographics

The targeted end users of our project would be AI developers. When AI is developed and trained, it consumes a lot of energy, but that also means it consumes a lot of capital. Reducing energy usage and cost in a growing AI economy is the goal for many developers and companies.

## Development Plan

As the entire team is returning from the previous semester, the learning curve to feel confident about working on this project is much less than usual. All members are proficient in Java and Python, so the only “unknown” technologies may be the functionality of Scalene, LibCST, and the chameleon servers we are working with. For all members to learn these, reviewing the documentation/repos and asking questions will comprise the process of learning. Specific benchmarks for ensuring that these technologies are learned are hard to pinpoint, but successful work being achieved each week using them should show that each person has learned.

## Semester Project Plan

## Sprint 1: Build Foundations for Refactoring and Profiling

- **Goal:** Lay the groundwork for ML-Driven Refactoring and prepare testing infrastructure.
- **Tasks:**
    - Set up basic LibCST parsing and small transformation examples.
    - Create example code snippets and datasets for training/testing ML models.
    - Set up server infrastructure to test Scalene modifications (CPU/GPU data collection).
    - Write initial unit tests for Scalene modifications (TDP fetching, profiling accuracy).

- **Deliverable:**
    - Working prototype that parses code with LibCST.
    - Basic Scalene modifications validated with unit tests.
- **User Stories:**
    - #9: As a software engineer developing algorithms dependent on machine learning, I want to be able to automatically refactor my code so I can reduce the energy consumption of these algorithms - Layne, Ahmed, Rob
    - #7: As a machine learning engineer, I want to be able to profile my code and so I can identify methods contributing to high energy consumption in joules. - Cristian, Jake, Ahmed, Layne, Rob

---

## Sprint 2: Develop ML-Driven Refactoring Core

- **Goal:** Start building the intelligent refactoring system (hardest part).
- **Tasks:**
    - Prototype a simple GNN (Graph Neural Network) or heuristic-based model that proposes refactors based on Scalene energy profiling.
    - Integrate ML model suggestions into the LibCST transformer pipeline.
    - Test early refactoring cases on small sample programs.
    - Continue building out Scalene profiling data (more sample runs, more data points).
- **Deliverable:**
    - End-to-end pipeline that can **analyze -> propose refactor -> rewrite** small pieces of code.
    - Documented testing results on a few hand-picked scripts.
- **User Stories:**

- #9: As a software engineer developing algorithms dependent on machine learning, I want to be able to automatically refactor my code so I can reduce the energy consumption of these algorithms - Layne, Ahmed, Rob
- #7: As a machine learning engineer, I want to be able to profile my code and so I can identify methods contributing to high energy consumption in joules. - Cristian, Jake, Ahmed, Layne, Rob

## Model selection:

### Option A: Template + Decision Model

**Template:**
A predefined code transformation pattern that can be applied under specific conditions. Basically rules to replace high cost code to lower cost code.

**Example Patterns**: loop-to-batch vectorization, replacing recursion with iteration, efficient tensor indexing
**Tools:** LibCST, Bowler

**Model:**
A decision model that used to decide where and which template to use for detected high energy patterns.

**Architecture**: GNN

**Related work:** AI-Driven Code Refactoring: Using Graph Neural Networks to Enhance Software Maintainability

**Take aways:**

1. Most of the refactoring is done by the **template**, the ML model is just an assistant
2. **Less data** needed, **high correctness**
3. Need to **high manual effort** for finding the energy pattern and designing the templates

### Option B: Generative Model + Validation

**Model:**
A generative model that learns code transformations and energy patterns from data and directly produces the refactored code.
**Architecture**: encoder-decoder, decoder only

**Take aways:**

1. The model did all the refactoring work, so a **large amount of validations** are needed for the correctness and energy improvement check.

2. More on LLM side but we train our own, so **large amount of data** needed and will be the best if the data is energy focused(I didn't found any yet)

| **Aspect** | **Decision Model** | **Generative Model** |
|---|---|---|
| Data requirement | Medium | Very High |
| Correctness | accurate | Hard to verify |
| flexibility | low | high |
| control | high | low |
| Refactor control by | Template | AI Model |
| Related work | Less | More |

---

## Sprint 3: UI and Extensive Testing Phase

- **Goal:** Make results accessible and validate the system.
- **Tasks:**
    - Develop a simple UI (could be a web dashboard) to upload code and view:
        - Scalene profiling outputs (energy use)
        - Suggested refactors
        - Before-and-after views
    - Expand testing:
        - Unit tests for Scalene and LibCST refactor modules.
        - Integration tests for full ML-driven pipeline.

        - Server load testing for Scalene profiling backend.

- **Deliverable:**
    - Web-based UI showing profiling data and refactor suggestions.
    - Full test suite run successfully.
- **User Stories:**
    - #9: As a software engineer developing algorithms dependent on machine learning, I want to be able to automatically refactor my code so I can reduce the energy consumption of these algorithms - Layne, Ahmed, Rob
    - #7: As a machine learning engineer, I want to be able to profile my code and so I can identify methods contributing to high energy consumption in joules. - Cristian, Jake, Ahmed, Layne, Rob
    - #12: As an environmentalist, I would like to see the carbon emissions of the algorithms I run in graphs and visuals through a web-based UI, so I can easily quantify the environmental impact. - Rob, Ahmed, Jake

---

## Sprint 4: Java Profiler (Meta Strobelight) Exploration

- **Goal:** Begin tackling Java profiling to extend system to multiple languages.
- **Tasks:**
    - Research Meta Strobelight and its capabilities.
    - Set up basic Java profiling experiments (energy/time usage).
    - Compare Java profiler output to Scalene methodology.
    - Plan how multi-language profiling might integrate into future versions.
- **Deliverable:**
    - Proof-of-concept Java profiler working locally.
    - Initial report comparing Java profiling results to Scalene's.
- **User Stories:**

- #9: As a software engineer developing algorithms dependent on machine learning, I want to be able to automatically refactor my code so I can reduce the energy consumption of these algorithms - Layne, Ahmed, Rob
- #7: As a machine learning engineer, I want to be able to profile my code and so I can identify methods contributing to high energy consumption in joules. - Cristian, Jake, Ahmed, Layne, Rob
- #10: As a machine learning engineer, I want to be able to profile my code in other languages like Java so I can expand the AI programs I can reduce energy for - Rob, Ahmed, Jake

---